

CLASSIFIED: INTERNAL CLUB USE ONLY

[▲] CYBERKAVACH 2.0

SOFTWARE REQUIREMENTS
SPECIFICATION (SRS)

Centralized Operations & Cyber Command Portal

DOCUMENT ID

CK2-SRS-2026-V2.0

REVISION DATE

June 16, 2026

AUTHOR

Lead Architect, CyberKavach

TARGET PORTAL

<https://github.com/IshanShastri30126/Cyberkavach2.0>

CLASSIFICATION

CONFIDENTIAL - Level 3
Operations

PLATFORM RELEASE

v2.0.0 (Production Core)

TABLE OF CONTENTS

1. Introduction & Scope

1.1 Document Purpose

1.2 System Overview

1.3 Technology Stack

2. User Roles & Permission Clearance Matrix

3. Detailed Functional Modules

3.1 Authentication & Registration HUD

3.2 Multilevel Approval Workflow Engine

3.3 Event Management & Roster Export

3.4 Team Manager & Roster Cloning

3.5 Attendance HUD Scanner Core

3.6 Cryptographic Certificate Custom Canvas & Issuance

3.7 Matrix Leaderboard & Gamification Milestone System

3.8 Real-Time Notifications (Socket.io) & Smart AI Chatbot

3.9 Audit Trail logging & Social Media Drafting

4. Database Schema Specification (Prisma Mapping)

5. Database Query Optimization & Caching Strategies

6. Non-Functional Requirements

1

1

1

1

2

3

3

3

4

4

5

5

6

6

7

8

11

13

1. INTRODUCTION & SCOPE

1.1 Document Purpose

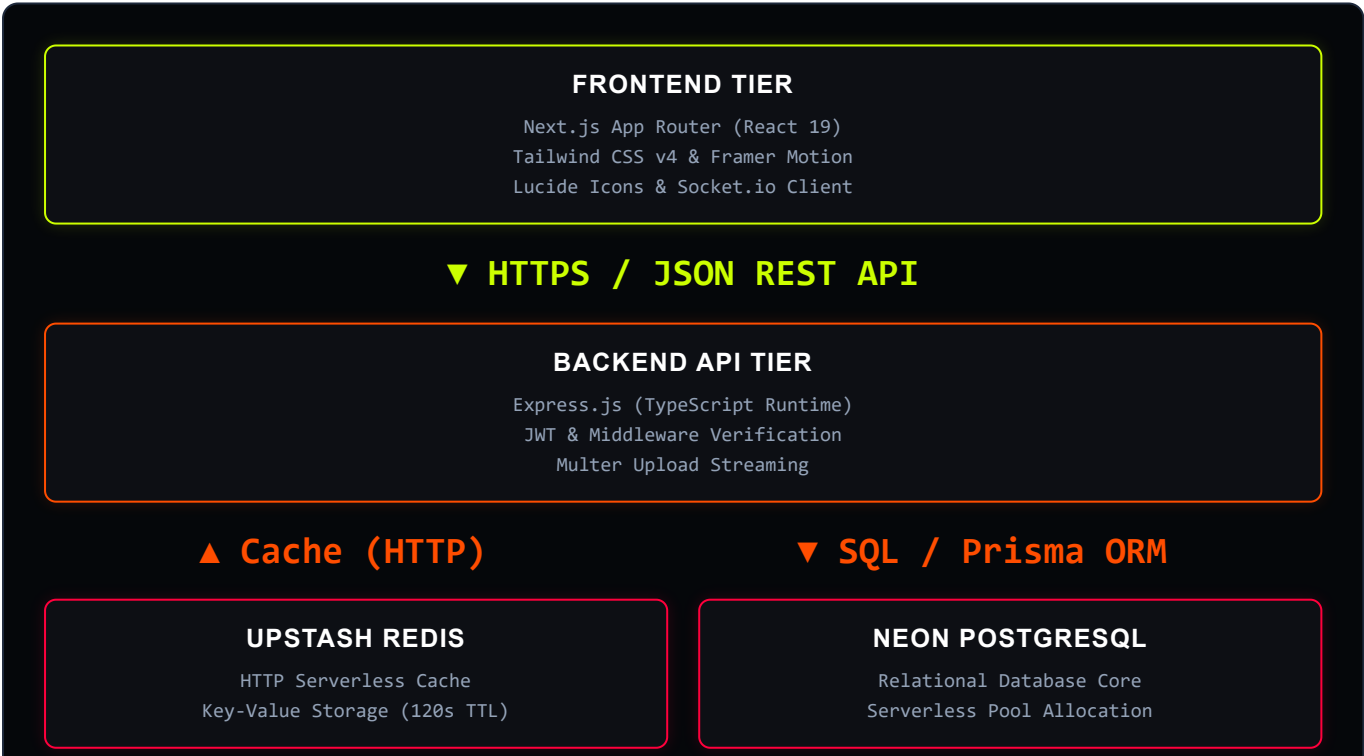
This Software Requirements Specification (SRS) establishes the baseline requirements, software architecture, data structures, and operational features of the **CyberKavach 2.0** operations portal. This document acts as the definitive design contract for students, faculty, developers, and administrators.

1.2 System Overview

CyberKavach 2.0 is a centralized operational hub designed specifically for cyber security clubs and technical student organizations. The platform handles end-to-end event organization, secure student and coordinator onboarding, biometric/QR code physical attendance logging, batch certificate compilation, multi-level approval hierarchies, newsletter editing, social media copy generation, real-time activity streaming, and analytical dashboard telemetry.

1.3 Technology Stack

The portal is architected using a modern decoupled framework tailored for high performance, ease of local development, and sub-millisecond database response times:



SpaceX Dashboard Aesthetic Reference

2. USER ROLES & PERMISSION CLEARANCE MATRIX

All frontend pages are styled in a dark matrix palette (#05070A background) containing glowing cyan, lime, orange, and red UI triggers, monospaced tech typography, responsive overlays, and seamless sliding transitions.

PAGE 2

2. USER ROLES & CLEARANCE MATRIX

The platform enforces strict role-based access control (RBAC). The following table defines the seven clearance tiers, mapped to their specific structural permissions in the code:

CLEARANCE ROLE	PRISMA ENUM	ASSIGNED SCOPE & PERMISSIONS
Faculty Sponsor	FACULTY	Highest authorization tier. Review and approve club budget/venue/event proposals; override approvals; award/deduct appreciation points; create certificate templates; manage club settings; approve student registrations.
Student Coordinator	STUDENT_COORDINATOR	Operational management. Create new event proposals; design rules; manage team rosters; submit social media posts for review; issue point logs to members; trigger attendance scanners.
Technical Team	TECH	Configuration and data exports. Read analytics; export CSV registration lists; configure event settings; verify server logs; debug certificates; query attendance databases.
Content Director	CONTENT	Draft and update event descriptions, rules, tags, code challenges, and announcements. Modify rules and guidelines text files.
Social Media Manager	SOCIAL_MEDIA	Draft and publish multi-channel social media posts (LinkedIn, Twitter, Instagram); organize newsletters; build marketing outlines.
Club Member	MEMBER	General member scope. View published events; register for hackathons/contests; create teams and invite peers; download team QR access codes; view point history and earned badges; download earned certificates.
External Guest	GUEST	Unapproved/Anonymous visitors. Register a new account; view public landing page and published event descriptions. Account remains in deactivated guest mode until approved by a FACULTY or STUDENT_COORDINATOR.

Clearance Warning

All API controllers gate requests via authorization middleware matching these enums. Any attempt to query higher-tier endpoints (e.g. PATCH /users/:id/role by a MEMBER) will trigger a standard HTTP 403 Forbidden response and log an audit trail entry.

3. DETAILED FUNCTIONAL MODULES

3.1 Authentication & Onboarding HUD

The onboarding module provides registration, password encryption, validation, and multi-tier account approval.

- **User Sign-Up Flow:** Registration requires email, password, full name, student ID, department, institute, and current semester. The default role assigned to any new registrant is GUEST, with `isApproved` explicitly set to `false`.
- **Approval Gating:** A registered guest cannot access the dashboard or join events. Their account must be verified and approved by a FACULTY or STUDENT_COORDINATOR via the clearance approvals page (triggers `isApproved: true`).
- **Google OAuth Integrations:** Optional Google Auth login is verified on the backend, checking for student domains and signing a JWT upon match.
- **Interactive UI States:** The login screen features a matrix console with sound triggers, scanning animation, and input checking with interactive red border vibrations on verification error.

3.2 Multilevel Approval Workflow Engine

Operations like publishing an event, spending a budget, or issuing certificates require systematic authorization.

- **Approval Requests:** A student coordinator creating a new event initiates a new database record in the `approval_requests` table (`status: PENDING`).
- **Multi-Level Clearance:** The system supports sequential approval steps. For events, it automatically defines a Step 1: level 1, target role FACULTY. The event remains in draft mode (`isDraft: true`, `isPublished: false`) and cannot be published until the step is marked APPROVED.
- **Escalation Rules:** Requests that have been in the queue for more than 48 hours without action flag a warning to student coordinators and highlight in red on the executive approvals view.
- **Notifications:** Creating an approval request automatically resolves active target roles (e.g. all active Faculty users) and creates batch database notifications alerting them to act.

3.3 Event Management & Roster Export

Coordinators can create and edit cyber-events, hackathons, capture-the-flag (CTF) contests, and training sessions.

- **Interactive Draft Builder:** Events are built via a multi-page stepper form including venue configuration, start/end date calculations, rules markdown block, tag selections, registration deadline, capacity, and min/max team size controls.
- **Asset Upload:** Uses Multer streaming to save poster images and PDF rulebooks locally to /uploads. Re-uploading automatically cleans up old files.
- **Public Visibility Gating:** Events remain hidden until `isPublished: true` and `isApproved: true`. When published, the backend registers an event reminder broadcast.
- **CSV Roster Exporting:** Faculty and Tech roles can download a compiled CSV spreadsheet of all event participants, containing full name, email, student ID, department, team name, and exact registration timestamp.

3.4 Team Manager & Roster Cloning

For team-based contests, members register by establishing or joining collaborative groups.

- **Team Code Generation:** Creating a team inserts a `Team` record and generates a unique, alphanumeric 6-digit invitation code (e.g., KAVACH-X7). Sharing this code allows peer members to join.
- **Peer invitations Gating:** Submitting a code to join checks if the user is already registered for that event, validates that the team is not full (does not exceed `maxTeamSize`), and checks if the member account is approved and active.
- **Roster Cloning (Cloning & Signing):** Users can clone a past successful team roster into a new event. The system queries the members of the old team, verifies their active status, and performs a bulk insert creating a new team with a fresh code, outputting a console feedback animation: *"Identity Cloned & Signed."*
- **QR Access Token:** The team leader can download a generated QR Code (pointing to the team ID) which represents their digital check-in passport.

3.5 Attendance HUD Scanner Core

Tracks student attendance live during workshops and CTFs using physical checkpoint scanning.

- **Dual-HUD scanner:** The client scanner accesses the camera stream to decode team or individual check-in QR codes. An animated neon scanning beam moves across the frame.
- **Scanning Telemetry Check:** Scanning a code makes a post request to `/api/attendance/scan`. The backend:
 1. Validates if the QR belongs to a valid team or user registered for the event.
 2. Checks the current time against the event's `startDate`. If the scan occurs after the start date, it flags the record as `isLate: true`.
 3. Ensures no duplicate check-ins exist for the same user in that event window.
- **Check-Out Flow:** The scanner can be toggled to Check-Out mode. This logs a matching `CHECK_OUT` record, which calculates total session duration.

3.6 Cryptographic Certificate Canvas & Issuance

Eliminates manual distribution of certificates by creating a dynamic, automated graphic rendering engine.

- **Canvas Designer:** Coordinators upload a clean PNG certificate template file. Using a drag-and-drop canvas, they map dynamic text fields (e.g. `recipientName`, `eventTitle`, `date`) to exact X/Y pixel coordinates and font-size options (saved as JSON in `certificate_templates`).
- **Batch Generation:** Uploading a recipient CSV triggers a batch queue. The server loads the template image, uses the `canvas` Node.js library to draw the recipient text at the mapped coordinates, generates a unique validation hash (checksum), and saves the compiled PNG/PDF.
- **Cryptographic Verification:** Each certificate prints a unique verification link containing the validation hash. Public visitors can enter this hash to verify the authenticity against database records.
- **Member Dossier:** Approved members can view and download their earned credentials directly from their personal certificate dashboard.

3.7 Matrix Leaderboard & Gamification Milestone System

Gamifies member contributions to incentivize engagement in cyber defense activities.

- **Leaderboard Ranking:** Displays active users ranked by total accumulated appreciation points. The top 3 members are presented on custom SpaceX-colored podiums with floating trophies.
- **Points Issuance:** Faculty and Coordinators can allocate points to members for categories like "Technical Contribution," "Event Management Excellence," or "Creative Contribution."
- **Milestone Badges:** The system automatically evaluates point milestones (e.g. 100 points unlocks "Cyber Sentinel," 500 points unlocks "Root Admin"). When a user crosses a milestone, a new record is added to user_badges and a congratulations banner displays.
- **biometric Profile Dossiers:** Clicking a member profile displays their activity dossier. Modifying profile details displays a biometric circular scan animation for security theme fidelity.

3.8 Real-Time Notifications (Socket.io) & Smart AI Chatbot

Ensures instant coordination and conversational operations support.

- **Real-Time Broadcasts:** Using Socket.io, operations notifications (approvals updates, point additions, badges earned) are pushed to the client immediately.
- **Bottom-Right Notification HUD:** Notifications slide in from the bottom-right. It contains an inline "Mark as Read" trigger and "Acknowledge & Redirect" buttons. It respects a local storage key (ck_disable_notification_popups) to allow users to mute popups.
- **Smart Chatbot:** An interactive console-style chatbot is available in the side HUD. It queries the local knowledge base, events rules, and venue directories to answer user questions about schedules, team requirements, and points.

3.9 Audit Trail logging & Social Media Drafting

Secures club operations against unauthorized adjustments and handles outreach.

- **System Audit Logs:** Critical actions (event creation, user status modifications, certificate template alterations, points deductions) trigger an insert to the audit_logs table, recording the timestamp, acting user ID, action type, and JSON metadata context.
- **Social Media Drafts:** Allows social media coordinators to write, edit, and queue posts for Twitter, LinkedIn, and Instagram based on event highlights. Posts must be approved by content directors before publishing.

4. DATABASE SCHEMA SPECIFICATION

The relational data model is maintained in PostgreSQL via the Prisma ORM. The entities, constraints, and relationships are defined below:

4.1 Core Enumerations

ENUM NAME	ALLOWED DATABASE VALUES	DESCRIPTION
Role	FACULTY, STUDENT_COORDINATOR, TECH, CONTENT, SOCIAL_MEDIA, MEMBER, GUEST	Role clearance levels.
ApprovalStatus	PENDING, UNDER_REVIEW, APPROVED, REJECTED	Workflow state.
ApprovalType	EVENT_PERMISSION, RESOURCE_VENUE, BUDGET, SOCIAL_MEDIA_POST, CONTENT_PUBLISH, CERTIFICATE_AUTH, EXTERNAL_COLLAB	Approval request types.
AttendanceType	CHECK_IN, CHECK_OUT	HUD scan categories.
CertificateStatus	PENDING, GENERATING, GENERATED, FAILED	Credential generation state.
NotificationType	APPROVAL_UPDATE, EVENT_REMINDER, TEAM_UPDATE, POINTS_RECEIVED, BADGE_EARNED, SYSTEM, ACCOUNT_APPROVED	Real-time alert routing.

4.2 User Entity Model (Table: users)

4.3 Event Entity Model (Table: events)

FIELD NAME	DATABASE TYPE	NULLABILITY	CONSTRAINTS & DEFAULT VALUE
id	VARCHAR(36)	NOT NULL	Primary Key (UUID)
passwordHash	VAR		
id	CHAR(36)	NOT NULL	Primary Key (UUID)
title	VAR		
studentId	VAR		
slug	CHAR(255)	NOT NULL	Unique Constraint Unique Index
startDate	TIMESTAMP	NOT NULL	-
isApproved	BOOLEAN	NOT NULL	Default: false
endDate	TIMESTAMP	NOT NULL	-
maxCapacity	INTEGER	NULLABLE	-
createdAt	TIMESTAMP	NOT NULL	Default: NOW()
minTeamSize	INTEGER	NULLABLE	Default: 1
maxTeamSize	INTEGER	NULLABLE	Default: 1
isPublished	BOOLEAN	NOT NULL	Default: false
isApproved	BOOLEAN	NOT NULL	Default: true
creatorId	VAR	NOT NULL	FK to users.id

4.4 Team & TeamMember Models (Tables: teams, team_members)

4.5 Attendance & Appreciation Models

FIELD NAME	DATABASE TYPE	NULLABILITY	KEY STATUS & FOREIGN KEY MAP
name	VARCHAR(100)	NOT NULL	-
id	VARCHAR(36)	NOT NULL	Primary Key (UUID)
leaderId	VARCHAR(36)	NOT NULL	FK to users.id
type	AttendanceType	NOT NULL	CHECK_IN or CHECK_OUT
isLate	BOOLEAN	NOT NULL	Default: false
userId	VARCHAR(36)	NOT NULL	FK to users.id (Index: attendance_userId_idx)
eventId	VARCHAR(36)	NOT NULL	FK to events.id (Index: attendance_eventId_idx)
teamId	VARCHAR(36)	NOT NULL	FK to teams.id (On Delete: Cascade)
id	VARCHAR(36)	NOT NULL	Primary Key (UUID)
points	INTEGER	NOT NULL	Negative value indicates a deduction
category	VARCHAR(100)	NOT NULL	Indexed category metadata
receiverId	VARCHAR(36)	NOT NULL	FK to users.id (Index: appreciation_receiverId_idx)
giverId	VARCHAR(36)	NOT NULL	FK to users.id (Index: appreciation_giverId_idx)

4.6 Approval System Models

5. DATABASE QUERY OPTIMIZATION & CACHING

PAGE 10

FIELD NAME	DATABASE TYPE	NULLABILITY	DESCRIPTION & RELATIONAL LINKS
Table: approval_requests			
id	VARCHAR(36)	NOT NULL	Primary Key (UUID)
type	ApprovalType	NOT NULL	Type of permission needed
status	ApprovalStatus	NOT NULL	Default: PENDING
requesterId	VARCHAR(36)	NOT NULL	FK to users.id

To prevent database N+1 bottlenecks and ensure lightning fast responsiveness under high load, CyberKavach 2.0 implements 16 specific optimizations across all routes:

5.1 Parallel Query Execution via Promise.all()

Sequential await operations block the event loop. The backend combines database reads in parallel wherever independent queries exist.

id	VARCHAR(36)	NOT NULL	Primary Key (UUID)
----	-------------	----------	--------------------

Code Optimization Blueprint (GET /api/analytics/club)

```
const [totalUsers, totalEvents, totalRegistrations, totalCertificates, roleDistribution, recentEvents, approvalStats] = await Promise.all([
  prisma.user.count(),
  prisma.event.count(),
  prisma.eventRegistration.count(),
  prisma.certificate.count(),
  prisma.user.groupBy({ by: ["role"], _count: { id: true } }),
  prisma.event.findMany({ take: 5, orderBy: { createdAt: "desc" } }),
  prisma.approvalRequest.groupBy({ by: ["status"], _count: { id: true } })
]);
```

By bundling 7 queries in a single Promise container, query execution latency drops from ~180ms to the slowest single query duration (~30ms).

5.2 Multi-Tier Upstash Redis Caching Strategy

High-traffic read endpoints utilize Upstash Serverless Redis caching using HTTP-based connections. This prevents socket exhaustion during local container restarts:

- 1. **Public Events Listing:** Cached at events:public:\${clubId} (TTL: 60s). Only unsearched first-page results are cached to maximize cache-hit rates.
- 2. **Club-Wide Analytics:** Cached at analytics:club (TTL: 60s) to limit expensive database count aggregations.
- 3. **Gamification Leaderboard:** Leaderboard periods are cached at appreciation:leaderboard:\${period} (TTL: 120s) because point tallies remain constant between checkpoints.

4. **Badge Milestones Dictionary:** Unlocked achievements directory is cached at `appreciation:badges` (TTL: 300s).

5. DATABASE QUERY OPTIMIZATION & CACHING

PAGE 11

5.3 Dynamic Cache Invalidation (Busting stale data)

5.4 Batch Database Operations (Validation & Insertion)

To maintain consistency, any write or update operations call `cacheInvalidate()` immediately before sending the API response, ensuring subsequent reads immediately load current records. Loading queries to perform checks is a critical bottleneck. The platform resolves validation in batch batches:

- **Teammate Batch Validation:** During team registration, rather than querying user approval status individually, the server runs a single `prisma.user.findMany({ where: { email: { in: emails } } })`. Email arrays are filtered and validated in-memory, avoiding round-trips.
- **Milestone Batch Evaluation:** Milestone checks run a single aggregate query of total points, pulls all eligible badges, and queries existing badges using `{ userId, badgeId: { in: eligibleBadges.map(b => b.id) } }`.
- **Bulk Notifications:** Triggering notifications to multiple stakeholders uses `sendBulkNotification`. The backend executes a single `prisma.notification.createMany()` write transaction, bypassing multiple individual writes.

5.5 ACID PostgreSQL Transactions (Data Integrity Guard)

Operations modifying multiple interrelated tables are wrapped in a database-level transaction. This ensures that if any single step fails, the entire transaction rolls back, preventing orphaned data:

Transactional Team Assembly Code Layout

```
const result = await prisma.$transaction(async (tx) => {
  const newTeam = await tx.team.create({ data: { name, teamCode, eventId, leaderId } });
  await tx.teamMember.createMany({
    data: [{ teamId: newTeam.id, userId: leaderId }, ...members]
  });
  await tx.eventRegistration.createMany({
    data: [{ userId: leaderId, eventId, teamId: newTeam.id }, ...registrations]
  });
  return newTeam;
});
```

This guarantees team metadata, user memberships, and event registrations are generated atomically.

5.6 Relational Database Index Mapping

The database schema includes explicit indices on fields heavily targeted by filter, sort, and search clauses:

- `@@index([clubId])` on `events`, `approval_requests`, `certificate_templates`, and `notifications` for namespace isolation.

6. NON-FUNCTIONAL REQUIREMENTS

PAGE 12

- `@@index([eventId])` and `@@index([userId])` on `attendance`, `teams`, and `event_registrations` to optimize relational joins.

6. NON-FUNCTIONAL REQUIREMENTS

- `@@index([certificateId])` and `@@index([createdAt])` on `approval_requests` to accelerate leaderboard ordering.

6.1 Security & Data Privacy

- **Cryptography:** All user passwords are encrypted using bcrypt hashing (salt rounds: 10). Cryptographic certificate verification links print unique SHA-256 checksums of the certificate PDF.
- **Session Gating:** Client requests require an HTTP Authorization Header: `Bearer <JWT_TOKEN>`. Tokens expire in 24 hours. Session storage is protected against XSS.
- **Input Sanitations:** Express router handles validation via Zod structures. Parameterized query templates constructed by Prisma prevent SQL injections.

6.2 Performance & Responsiveness

- **Response Time Thresholds:** High-frequency read queries (homepage list, profile dossiers, dashboard counters) must return in under 50ms using Upstash Redis. Write endpoints (event check-in, points allocation) must resolve in under 150ms.
- **File Storage Management:** Multer uploads restrict files to 5MB and validate extensions (e.g. only `.png`, `.jpg`, `.pdf` are accepted).

6.3 System Scalability & Maintainability

- **Modular Abstraction:** The monorepo uses clear separation of concerns: routes manage endpoints and request validations; database queries are handled through Prisma client helper modules.
- **Environment Configs:** Port bindings, database URLs, Upstash tokens, and session secrets are managed via local `.env` settings files.

6.4 Printing & Compile Layouts

The SRS document structure is configured with page breaks and margin alignments to export to PDF cleanly, ensuring structural tables, code blocks, and diagrams are not split mid-page during headless Chrome execution.